

Counting Sort Algorithm – Notes, Example, and Implementation

Prepared by: Kinza Afzal

Date: November 3, 2025

1. Introduction

Counting Sort is a non-comparison-based sorting algorithm that sorts integers within a known range. It works by counting the frequency of each element and then determining their position in the output array.

2. Logic & Steps

1. Find the maximum value in the input array (`max_val`).
2. Initialize a `count` array of size `max_val + 1` with zeros.
3. Count the occurrences of each element in the input array.
4. Compute cumulative counts to determine the final positions.
5. Build the sorted `output` array by placing each element at its correct position (right to left for stability).
6. Copy the output array back to the original array.

3. C++ Code Implementation

Listing 1: Counting Sort Implementation in C++

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

void countingSort(vector<int>& arr) {
    int n = arr.size();
    if (n == 0) return;

    int max_val = *max_element(arr.begin(), arr.end());

    vector<int> count(max_val + 1, 0);
    vector<int> output(n);

    // Step 1: Count occurrences
    for (int i = 0; i < n; i++)
        count[arr[i]]++;

    // Step 2: Cumulative sum
```

```

    for (int i = 1; i <= max_val; i++)
        count[i] += count[i - 1];

    // Step 3: Build output (stable)
    for (int i = n - 1; i >= 0; i--) {
        output[count[arr[i]] - 1] = arr[i];
        count[arr[i]]--;
    }

    // Step 4: Copy to original
    for (int i = 0; i < n; i++)
        arr[i] = output[i];
}

int main() {
    vector<int> arr = {4, 2, 2, 8, 3, 3, 1};
    countingSort(arr);

    cout << "Sorted Array: ";
    for (int x : arr) cout << x << " ";
    cout << endl;
    return 0;
}

```

4. Dry Run Example

Input

$$arr = [4, 2, 2, 8, 3, 3, 1]$$

Step 1: Find Maximum Element

$$max_val = 8$$

Step 2: Initialize Count and Output Arrays

$$count = [0, 0, 0, 0, 0, 0, 0, 0] \quad output = [0, 0, 0, 0, 0, 0, 0]$$

Step 3: Count Frequency of Each Element

i	arr[i]	count array after update
0	4	[0,0,0,0,1,0,0,0]
1	2	[0,0,1,0,1,0,0,0]
2	2	[0,0,2,0,1,0,0,0]
3	8	[0,0,2,0,1,0,0,1]
4	3	[0,0,2,1,1,0,0,1]
5	3	[0,0,2,2,1,0,0,1]
6	1	[0,1,2,2,1,0,0,1]

Final count array: [0, 1, 2, 2, 1, 0, 0, 1]

Step 4: Cumulative Count

[0, 1, 3, 5, 6, 6, 6, 6, 7]

Step 5: Build Output Array

i	arr[i]	count before	position	output update
6	1	1	0	[1,0,0,0,0,0,0]
5	3	5	4	[1,0,0,0,3,0,0]
4	3	4	3	[1,0,0,3,3,0,0]
3	8	7	6	[1,0,0,3,3,0,8]
2	2	3	2	[1,0,2,3,3,0,8]
1	2	2	1	[1,2,2,3,3,0,8]
0	4	6	5	[1,2,2,3,3,4,8]

Final Sorted Array: [1, 2, 2, 3, 3, 4, 8]

5. Time and Space Complexity

Best Case: $O(n + k)$

Average Case: $O(n + k)$

Worst Case: $O(n + k)$

Space Complexity: $O(n + k)$

Stable: Yes

In-place: No

Where:

n = number of elements, k = range of input (max - min)

6. LeetCode Example: Sort Colors (Problem #75)

Problem Statement

Given an array `nums` with 0s, 1s, and 2s, sort them in-place so that all 0s come first, then 1s, then 2s.

Counting Sort Solution

Listing 2: LeetCode 75: Sort Colors Using Counting Sort

```
class Solution {
public:
    void sortColors(vector<int>& nums) {
        int n = nums.size();
        int max_val = *max_element(nums.begin(), nums.end());
        vector<int> count(max_val + 1, 0);
        vector<int> output(n, 0);

        // Count frequencies
        for (int i = 0; i < n; i++)
```

```

        count[nums[i]]++;

    // Cumulative sum
    for (int i = 1; i <= max_val; i++)
        count[i] += count[i - 1];

    // Build sorted output
    for (int i = n - 1; i >= 0; i--) {
        output[count[nums[i]] - 1] = nums[i];
        count[nums[i]]--;
    }

    // Copy back
    for (int i = 0; i < n; i++)
        nums[i] = output[i];
}
};

```

Example Run

$nums = [2, 0, 2, 1, 1, 0]$

After sorting:

$nums = [0, 0, 1, 1, 2, 2]$

7. Advantages

- Fast for small range integers.
- Stable and non-comparison-based.

8. Disadvantages

- Not suitable for large ranges of numbers.
- Uses extra space proportional to the maximum element.

9. Conclusion

Counting Sort is an efficient and stable algorithm when the range of input values is small relative to the number of elements. It forms the basis for more advanced algorithms like Radix Sort.